



INSTITUTO POLITECNICO NACIONAL

Escuela Superior de Computo

COMPILADORES

Práctica 4 – Analizador Léxico LL(1)

Profesora: ALBORTANTE MORATO CECILIA

Alumnos: Gutiérrez Espinosa Jordan Gabriel
Ricardo Bisogno Gandarilla

Grupo: 5CM2

Ciclo: 2022-2023(2)



Introducción	1
DESARROLLO	2
Resultados	8
Referencias	12

INTRODUCCION

El analizador léxico es la primera fase del proceso de análisis sintáctico de un compilador, cuyo objetivo es descomponer el código fuente en una secuencia de tokens. El analizador léxico utiliza expresiones regulares para reconocer patrones léxicos en el código fuente, como identificadores, palabras clave, literales, símbolos y operadores.

El analizador léxico LL(1) es un tipo de analizador sintáctico predictivo de descendente izquierdo de un solo símbolo de avance. El término LL(1) se deriva de "Left-to-right, Leftmost derivation, 1 symbol lookahead" (De izquierda a derecha, deriva más a la izquierda, con anticipación de 1 símbolo). Esto significa que el analizador procesa el código fuente de izquierda a derecha y construye una derivación más a la izquierda, mientras anticipa el siguiente símbolo para tomar decisiones.

El analizador léxico LL(1) se basa en una gramática libre de contexto LL(1), que debe cumplir con las siguientes características:

Debe ser no ambigua: Cada cadena de entrada debe tener una única derivación en la gramática.

Debe de respetar las reglas de una operación: segira las sintaxis de como debe de ser la operacional.

Debe de tener los operadores existentes: debe de respetar los operadores que se tegan.

Debe ser LL(1): El analizador debe poder determinar el siguiente paso a tomar mirando únicamente el siguiente símbolo de entrada y la parte más a la izquierda de la pila de análisis.

La principal ventaja del analizador léxico LL(1) es su simplicidad y eficiencia, ya que no requiere la construcción de tablas de análisis sintáctico ni backtracking (retroceso). Sin embargo, su limitación principal es que solo puede analizar gramáticas LL(1), lo cual restringe su uso a un subconjunto de gramáticas más simples como se muestra en la Figura 1.



Figura 1, Base Analizador LL(1).

DESARROLLO

AnalizadorSintacticoLL1.java

En esta sección del código, se realiza la inicialización de la tabla de análisis sintáctico y se solicita al usuario ingresar la cadena a analizar. La variable "resp" se declara para almacenar la respuesta del usuario en caso de querer comprobar otra cadena después de la ejecución del programa.

La función "*inicializarTablaParsing*" se llama para inicializar la tabla de análisis sintáctico LL(1). Esta tabla es un objeto Map que almacena las producciones gramaticales para cada combinación de símbolos no terminales y terminales. Dentro del bucle "*while (true)*", se solicita al usuario ingresar la cadena a analizar mediante la lectura de la entrada del usuario utilizando "*System.console().readLine()*". La cadena ingresada se guarda en la variable "*preliminarText*".

Después de eliminar los espacios en blanco, se agrega el símbolo de fin de cadena '\$' al final de la cadena para indicar el final de la entrada.

Este resumen se centra en la inicialización de la tabla de análisis sintáctico y el procesamiento de la cadena de entrada por parte del usuario. Para comprender completamente el funcionamiento del programa y el análisis sintáctico LL(1), es necesario revisar el código completo y estudiar los conceptos relacionados con la gramática y las tablas de análisis sintáctico LL(1).

```
package clases;
import java.util.HashMap;
import java.util.Map;

public class AnalizadorSintacticoLL1 {
    private static final Map<String, Map<String, String>> tablaParsing = new HashMap<>();

    Run | Debug
    public static void main(String[] args) {
        String resp;
        // Inicializar la tabla de análisis
        inicializarTablaParsing();
        while (true) {
            String preliminarText;
            // Cadena a analizar
            String cadena;
            System.out.print("\n Por favor ingrese la cadena a analizar: ");
            preliminarText = System.console().readLine();
            cadena = "";
            for (int i = 0; i < preliminarText.length(); i++) {
                if (preliminarText.charAt(i) == ' ') {
                    continue;
                }
                cadena += preliminarText.charAt(i);
            }
            cadena += '$';
        }
    }
}
```

Figura 2, Comienzo del análisis.

Se crea una instancia de la clase "Pila" llamada "pila" mediante la instrucción "*Pila pila = new Pila();*". A continuación, se agregan dos símbolos a la pila: el símbolo de fin de cadena '\$' y el símbolo inicial 'E'. Esto se realiza mediante las llamadas a los métodos "*Insertar('\$')*"

y "*Insertar('E')*" respectivamente. Se inicializa la variable "*indice*" con el valor 0, que se utiliza para recorrer la cadena de entrada. Las variables "*simboloTope*" y "*simboloActual*" se declaran para almacenar los símbolos que se compararán durante el análisis sintáctico.

Se inicializa la variable "*cadenaValida*" como verdadera, indicando que la cadena de entrada se considera válida hasta que se encuentre una discrepancia durante el análisis. A continuación, se entra en un bucle "*while*" que se ejecuta mientras la pila no esté vacía. Dentro del bucle, se imprime el contenido de la pila mediante el método "*verComponentes()*" de la instancia "*pila*".

Se extrae el símbolo en el tope de la pila utilizando el método "*extraer()*" y se almacena en la variable "*simboloTope*".

Luego se verifica si el símbolo actual en la cadena de entrada es un dígito o una letra minúscula. Si no lo es, se asigna el símbolo actual de la cadena a la variable "*simboloActual*" utilizando el método "*charAt(indice)*". En caso contrario, si el símbolo actual es un dígito o una letra minúscula, se utiliza un bucle "*while*" para avanzar en la cadena de entrada hasta encontrar un símbolo que no sea un dígito ni una letra minúscula. Durante este proceso, se incrementa el valor de "*indice*" para recorrer la cadena.

Una vez se obtiene el símbolo actual, se verifica si el símbolo en el tope de la pila o en la cadena es un espacio en blanco utilizando el método "*isWhitespace()*". Si es un espacio en blanco, se extrae nuevamente un símbolo de la pila utilizando el método "*extraer()*" y se asigna a "*simboloTope*" o se adelanta el término en la cadena.

```
// Crear la pila y agregar el símbolo inicial
Pila pila = new Pila();
pila.Insertar(valor:'$');
pila.Insertar(valor:'E');

// Inicializar el índice para recorrer la cadena
int indice = 0;
char simboloTope;
char simboloActual;
// Realizar el análisis sintáctico
boolean cadenaValida = true;

while (!pila.PilaVacía()) {
    System.out.println(x:" Pila: ");
    pila.verComponentes();
    simboloTope = pila.extraer();
    if(!Character.isDigit(cadena.charAt(indice))&&!Character.isLowerCase(cadena.charAt(indice)))
    {
        simboloActual = cadena.charAt(indice);
        if (Character.isWhitespace(simboloActual))
        {
            indice++;
            simboloActual = cadena.charAt(indice);
        }
    }
    else
    {
        while(Character.isDigit(cadena.charAt(indice+1))||Character.isLowerCase(cadena.charAt(indice+1)))
        {
            indice++;
        }
        simboloActual='i';
    }
    if (Character.isWhitespace(simboloTope))
    {
        simboloTope = pila.extraer();
    }
}
```

Figura 3, extracción de pila y carácter para comparar.

A continuación, se verifica si el símbolo en el tope de la pila es un símbolo terminal utilizando el método *esTerminal(simboloTope)* o si coincide con el símbolo actual de la cadena de entrada.

Si el símbolo en el tope de la pila coincide con el símbolo actual, se avanza al siguiente símbolo de la cadena incrementando el valor de *indice* y se extrae el símbolo de la pila utilizando el método *extraer()*. Esto indica que los símbolos coinciden y se continúa con el siguiente símbolo a analizar. En caso de que los símbolos no coincidan, se establece *cadenaValida* como falso y se sale del bucle utilizando *break*. Esto indica que la cadena de entrada no es válida según la gramática definida.

Si el símbolo en el tope de la pila no es un símbolo terminal o no coincide con el símbolo actual, se obtiene la producción correspondiente de la tabla de análisis sintáctico utilizando el método "get()" del objeto "tablaParsing". La producción se almacena en la variable "produccion". Se imprime la información de la producción en la consola utilizando la instrucción "System.out.println("Caracter LL1: "+produccion+"\n");".

Si la producción obtenida no es nula, se verifica si es diferente de "e" (epsilon). Si no es epsilon, se recorre la producción de manera inversa y se insertan los símbolos en la pila utilizando el método "Insertar(produccion.charAt(i))". En caso de que la producción sea nula, se establece "cadenaValida" como falso y se sale del bucle utilizando "break".

Esta parte del código es crucial para el análisis sintáctico LL(1) ya que compara los símbolos en la pila con los símbolos de la cadena de entrada y realiza las acciones correspondientes según la tabla de análisis sintáctico, como avanzar en la cadena, extraer símbolos de la pila o insertar nuevos símbolos en la pila. También se encarga de verificar la validez de la cadena de entrada según la gramática definida.

```
System.out.println("\n Argumento: " + simboloTope + " caracter " + simboloActual+"\n");
if (esTerminal(simboloTope)||simboloTope == simboloActual)
{
    if (simboloTope == simboloActual) {
        // Coinciden los símbolos
        indice++;
        simboloTope = pila.extraer();
    } else {
        cadenaValida = false;
        break;
    }
}
else
{
    String produccion = tablaParsing.get(Character.toString(simboloTope)).get(Character.toStrin
    System.out.println("Caracter LL1: "+produccion+"\n");
    if (produccion != null) {
        if (!produccion.equals(anObject:"e"))
        {
            for (int i = produccion.length() - 1; i >= 0; i--) {
                pila.Insertar(produccion.charAt(i));
            }
        }
        else
        {
            cadenaValida = false;
            break;
        }
    }
}
```

Figura 4, comparación de carácter de la pila y la cadena.

Se utiliza una estructura condicional "if" para evaluar dos condiciones: si la variable "cadenaValida" es verdadera y si el índice alcanza la longitud de la cadena. Esto significa que

la cadena se considera válida si se han procesado todos los símbolos de la cadena y no se ha detectado ninguna discrepancia durante el análisis sintáctico.

Si ambas condiciones se cumplen, se imprime en la consola el mensaje "La cadena es válida." En caso contrario, si la cadena no cumple alguna de las condiciones, se imprime en la consola el mensaje "La cadena no es válida."

Luego, se solicita al usuario si desea comprobar otra cadena. ¿Se muestra el mensaje "Desea comprobar otra cadena? En caso de no digite n, en caso de si presione cualquier tecla:" y se lee la respuesta del usuario mediante la instrucción `resp = System.console().readLine();`.

Si la respuesta del usuario es "n" o "N", se utiliza una estructura `"break"` para salir del bucle principal y finalizar el programa. Esta parte del código se encarga de mostrar el resultado de la validación de la cadena y brinda al usuario la opción de comprobar otra cadena o finalizar la ejecución del programa.

```
// Verificar si la cadena es válida
if (cadenaValida && indice == cadena.length()) {
    System.out.println(x:"La cadena es válida.");
} else {
    System.out.println(x:"La cadena no es válida.");
}
System.out.print(s:"\n Desea comprobar otra cadena? En caso de no digite n, en caso de si presione
resp = System.console().readLine();
if (resp.equals(anObject:"n") || resp.equals(anObject:"N")) {
    break;
}
}
```

Figura 5, Validación y recursividad.

Y por último se encuentra la función `inicializarTablaParsing()`, la cual se encarga de inicializar una tabla de análisis utilizada en el análisis sintáctico LL(1). Esta tabla es implementada utilizando una estructura de datos Map de la biblioteca Java.

La tabla de análisis se construye asignando producciones a cada combinación de símbolos no terminales y terminales. A continuación, se describe cada una de las producciones definidas en la tabla (se tomó la libertad de colocar como $T'=L$ y $E'=O$ para mayor comodidad al código):

Para el no terminal E, se definen dos producciones:

Si el terminal es 'i' o '(', la producción es "T O".

Para el no terminal O, se definen cuatro producciones:

Si el terminal es '+', la producción es "+ T O".

Si el terminal es '-', la producción es "- T O".

Si el terminal es ')' o '\$', la producción es "e", que representa la producción nula (epsilon).

Para el no terminal T, se definen dos producciones:

Si el terminal es 'i' o '(', la producción es "F L".

Para el no terminal L, se definen siete producciones:

Si el terminal es '*', la producción es "* F L".

Si el terminal es '/', la producción es "/ F L".

Si el terminal es '+', '-', ')', '\$', o 'i', la producción es "e", que representa la producción nula (epsilon).

Para el no terminal F, se definen dos producciones:

Si el terminal es 'i', la producción es "i".

Si el terminal es '(', la producción es "(E)".

Cada producción se define como una cadena de caracteres que representa la secuencia de símbolos a ser insertados en la pila durante el análisis sintáctico.

Las producciones se asignan a cada combinación de no terminales y terminales utilizando un Map anidado. Los no terminales (E, O, T, L, F) son utilizados como claves en el primer nivel del Map, y los terminales ('i', '(', ')', '+', '-', '*', '/', '\$') son utilizados como claves en el segundo nivel del Map. Las producciones correspondientes se obtienen accediendo a los valores del Map.

```
private static void inicializarTablaParsing() {  
    // E -> TE'  
    Map<String, String> E = new HashMap<>();  
    E.put(key:"i", value:"T O");  
    E.put(key:"(", value:"T O");  
    tablaParsing.put(key:"E", E);  
  
    // E' -> +TE' | ε  
    Map<String, String> O = new HashMap<>();  
    O.put(key:"+", value:"+ T O");  
    O.put(key:"- ", value:"- T O");  
    O.put(key:")", value:"e");  
    O.put(key:"$", value:"e");  
    tablaParsing.put(key:"O", O);  
  
    // T -> FT'  
    Map<String, String> T = new HashMap<>();  
    T.put(key:"i", value:"F L");  
    T.put(key:"(", value:"F L");  
    tablaParsing.put(key:"T", T);  
  
    // T' -> *FT' | ε L=T'  
    Map<String, String> L = new HashMap<>();  
    L.put(key:"*", value:"* F L");  
    L.put(key:"/", value:"/ F L");  
    L.put(key:"+", value:"e");  
    L.put(key:"- ", value:"e");  
    L.put(key:")", value:"e");  
    L.put(key:"$", value:"e");  
    L.put(key:"i", value:"e");  
    tablaParsing.put(key:"L", L);  
}
```

Figura 6, tabla LL(1).



Resultados

El proceso comienza solicitando al usuario que ingrese una cadena a analizar. En este caso, se proporciona la cadena "2+3+4". A continuación, se inicializa una pila con el símbolo inicial E y el marcador de fin de cadena \$.

Luego, se realiza el análisis sintáctico utilizando un bucle while. En cada iteración del bucle, se extrae el símbolo en la parte superior de la pila y se compara con el siguiente símbolo de la cadena de entrada. Se imprimen mensajes para mostrar el argumento y el carácter actual que se están analizando.

Si el símbolo en la pila es un terminal o coincide con el símbolo actual, se realiza una acción específica. Si coinciden, se avanza al siguiente carácter de la cadena y se extrae el símbolo de la pila. Si no coinciden, la cadena no es válida y se rompe el bucle.

Después de realizar el análisis sintáctico, se verifica si la cadena es válida verificando que la cadena haya sido completamente analizada (el índice sea igual a la longitud de la cadena) y que la variable cadenaValida sea verdadera. Se imprime un mensaje indicando si la cadena es válida o no.

Finalmente, se le pregunta al usuario si desea comprobar otra cadena. Si responde con "n" o "N", se sale del bucle y el programa finaliza.



Práctica 4 - Análisis sintáctico LL(1)
Por favor ingrese la cadena a analizar: 2+3+4

Pila:

E

\$

Argumento: E caracter i

Caracter LL1: T O

Pila:

T

O

\$

Argumento: T caracter i

Caracter LL1: F L

Pila:

F

L

O

\$

Argumento: F caracter i

Caracter LL1: i

Argumento: i caracter i

Pila:

L

O

\$

Argumento: L caracter +

Caracter LL1: e

Pila:

O

\$

Argumento: O caracter +

Caracter LL1: + T O

Pila:

+

T

O

\$

Argumento: + caracter +

Pila:

```
0
$

Argumento: i caracter i
Pila:
L
0
$

Argumento: L caracter $
Caracter LL1: e
Pila:
0
$

Argumento: 0 caracter $
Caracter LL1: e
Pila:
$

Argumento: $ caracter $
La cadena es válida.

Desea comprobar otra cadena? En caso de no digite n, en caso de si presione cualquier tecla: █
```

Figura 7, Funcionamiento del analizador, palabra aceptada.

Y aquí esta cuando la cadena esta equivocada.

```
Por favor ingrese la cadena a analizar: (a+b+c
Pila:
E
$

Argumento: E caracter (
Caracter LL1: T O
Pila:
T
O
$

Argumento: T caracter (
Caracter LL1: F L
Pila:
F
L
O
$

Argumento: F caracter (
Caracter LL1: ( E )
Pila:

Argumento: L caracter $
Caracter LL1: e
Pila:
O
)
L
O
$

Argumento: O caracter $
Caracter LL1: e
Pila:
)
L
O
$

Argumento: ) caracter $
La cadena no es válida.
```

Figura 8, Cadena no aceptada.



Referencias

M. A. Hicks, "An introduction to parsing: Top-down and bottom-up," IEEE Potentials, vol. 23, no. 4, pp. 6-10, Jul./Aug. 2004, <https://ricardogeek.com/tipos-de-parsers-en-un-compilador/>.

S. Rajkumar and S. Chakraborty, "A survey of parsing techniques," in 2012 IEEE International Conference on Advanced Communication Control and Computing Technologies (ICACCCT), pp. 1229-1234, Jun. 2012, <https://www.cs.ox.ac.uk/files/219/parsing.pdf>

A. V. Aho, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Reading, MA, USA: Addison-Wesley Publishing Company, Inc., 2006 <https://www.dbscience.org/wp-content/uploads/2020/03/ALSUdragonbookcompilers.pdf>.